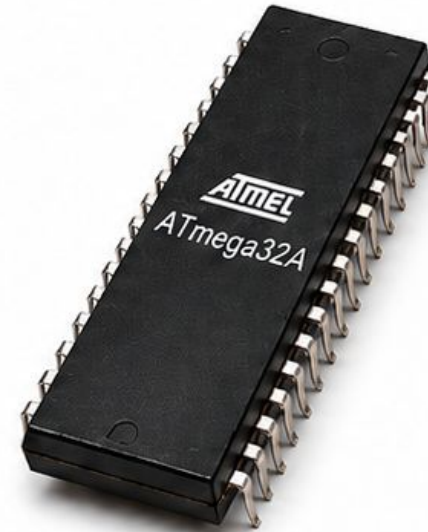


Timers in ATmega32A Embedded Systems

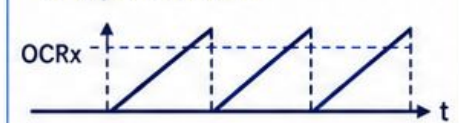
Normal Mode, CTC, PWM, WDT, ICU, and Timer Registers



Overflow (TOP)



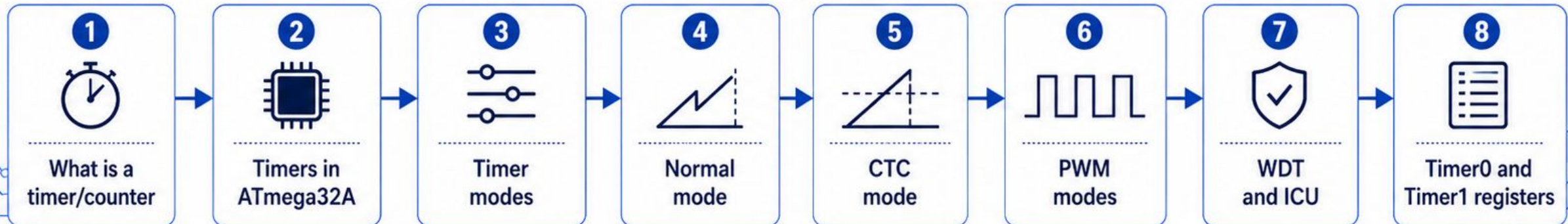
Compare Match



PWM Output



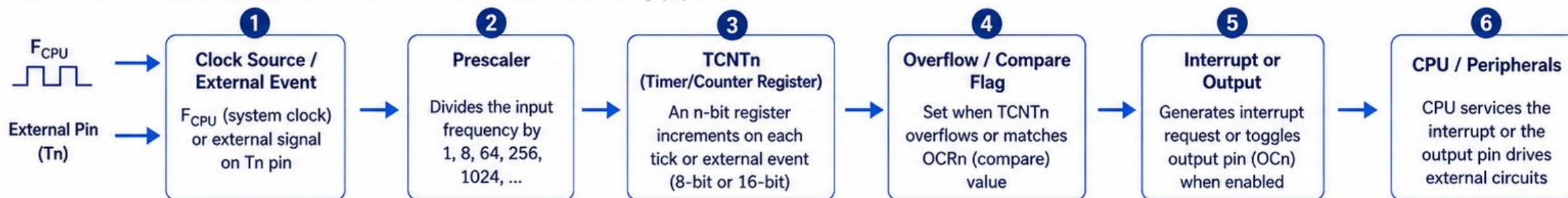
Session Roadmap



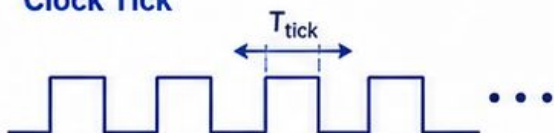
What Is a Timer / Counter?



In ATmega32A, a **timer** counts internal clock ticks, while a **counter** counts external events. Both use the same hardware and follow the same counting pipeline.



Clock Tick



Timing Relationships

$$T_{\text{tick}} = \frac{\text{Prescaler}}{F_{\text{CPU}}}$$

$$\text{Timer frequency} = \frac{F_{\text{CPU}}}{\text{Prescaler}}$$

F_{CPU} : System clock frequency (Hz)
 T_{tick} : Time between two timer ticks (s)
 Prescaler : Division factor selected for the timer
 TCNTn : Timer/Counter register (n = 0, 1, 2)



Timer Mode

TCNTn is incremented by internal clock ticks (after prescaler). Used for timing delays, time measurement, PWM, etc.



Counter Mode

TCNTn is incremented by external events on the Tn pin (after optional noise canceller and prescaler). Used for counting pulses, edges, etc.



Prescaler

Reduces the input frequency to a manageable rate for counting. Common settings: 1, 8, 64, 256, 1024.



Timer Interrupt

Triggered on overflow or compare match. Enables the CPU to respond to timing events without polling.

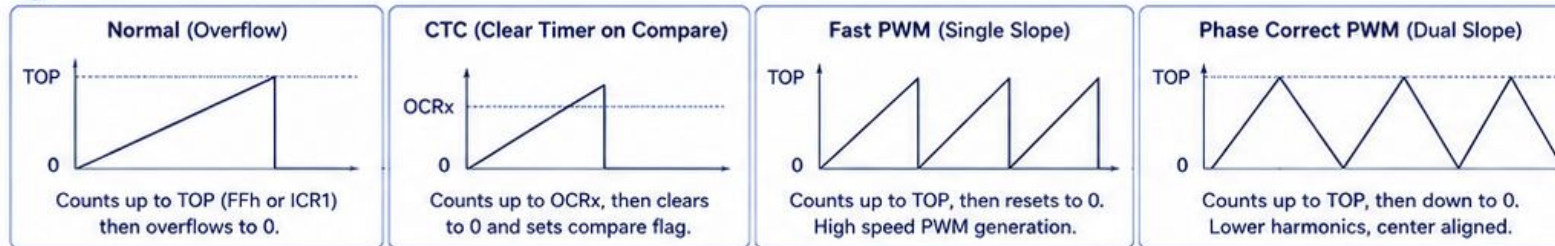
Timers in ATmega32A

Three flexible timers/counters for timing, counting, capture, compare, and PWM generation

Timer comparison

Timer	Resolution (Counter size)	Main Modes	Output Compare pins	Special notes
Timer0	8-bit	- Normal - CTC - Fast PWM - Phase Correct PWM	OC0 → PB3 (TCCR0, OCR0)	8-bit timer/counter - Prescaler: 1, 8, 64, 256, 1024 - Overflow interrupt (TOV0) - Compare Match interrupt (OCF0)
Timer1	16-bit	- Normal - CTC - Fast PWM - Phase Correct PWM - Input Capture	OC1A → PD5 OC1B → PD4 ICP1 → PD6 (TCCR1A/B, OCR1A/B, ICR1)	16-bit timer with Input Capture - Prescaler: 1, 8, 64, 256, 1024 - Input Capture on ICP1 (PD6) - Overflow (TOV1) & Compare Match (OCF1A/B) interrupts
Timer2	8-bit	- Normal - CTC - Fast PWM - Phase Correct PWM	OC2 → PD7 (TCCR2, OCR2)	8-bit timer/counter - Prescaler: 1, 8, 32, 64, 128, 256, 1024 - Asynchronous option with 32.768 kHz watch crystal (on TOSC1/2) - Overflow (TOV2) & Compare Match (OCF2) interrupts

Mode overview (counter behavior)



ATmega32A mini map

(XCK/T0)	PB0	1		40	PA0	(ADC0)
(T1)	PB1	2		39	PA1	(ADC1)
(INT2/AIN0)	PB2	3		49	PA2	(ADC2)
(OC0/T1)	PB3	4		48	PA3	(ADC3)
(SS)	PB4	5		37	PA4	(ADC4)
(MOSI)	PB5	6		36	PA5	(ADC5)
(MISO)	PB6	7		36	PA6	(ADC6)
(SCK)	PB7	8		35	PA7	(ADC7)
RESET		9		38	AREF	
VCC		10		33	GND	
GND		11		34	AVCC	
XTAL2		12		34	PC7	(TOSC2)
XTAL1		13		34	PC6	(TOSC1)
(RXD)	PD0	14		38	PC5	(TDI)
(TXD)	PD1	15		30	PC4	(TDO)
(INT0)	PD2	16		39	PC3	(TMS)
(INT1)	PD3	17		22	PC2	(TCK)
(OC1B)	PD4	18		28	PC1	(SDA)
(OC1A)	PD5	19		29	PC0	(SCL)
(ICP1)	PD6	20		21	PD7	(OC2)

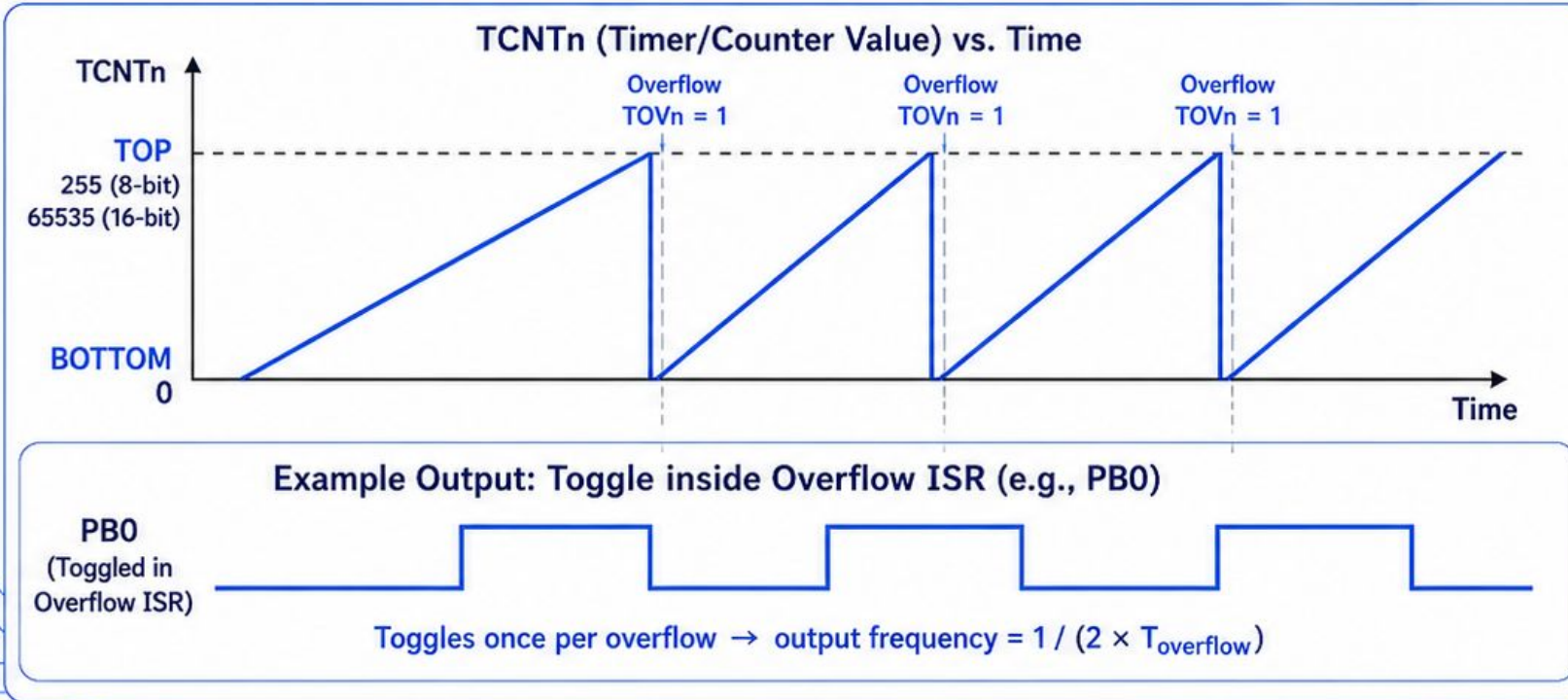
□ Timer0 (OC0) □ Timer1 (OC1A/OC1B/ICP1) □ Timer2 (OC2)

Key takeaways

- Use **Timer0** for simple timing, delays, and 8-bit PWM.
- Use **Timer1** for precise 16-bit timing, input capture, and advanced PWM.
- Use **Timer2** for 8-bit PWM/timing with optional asynchronous clock (low power RTC).
- All timers can generate interrupts on overflow and compare match.

Normal Mode: Overflow-Based Timing

In Normal mode, the timer/counter counts from **BOTTOM** (0) to **TOP** (255 for 8-bit or 65535 for 16-bit), then overflows to 0 and sets **TOVn**.



i How Overflow Interrupts Work

- When the counter overflows, **TOVn** in **TIFR** is set.
- If **TOIE_n** (Timer/Counter Overflow Interrupt Enable bit) in **TIMSK** is set, an interrupt request is generated.
- The interrupt is executed only if the global interrupt enable bit (**I-bit**) in **SREG** is set.

🕒 Overflow Period

$$T_{\text{overflow}} = \frac{(\text{MaxCount} + 1) \times \text{Prescaler}}{F_{\text{CPU}}}$$

- MaxCount = 255 (8-bit) or 65535 (16-bit)
- Prescaler = selected clock prescaler
- F_{CPU} = CPU clock frequency

✅ Why It Matters

- Simple and efficient timing source
- Ideal for delays and periodic tasks
- Generates consistent interrupts at fixed intervals

Normal Mode: Three Timing Cases

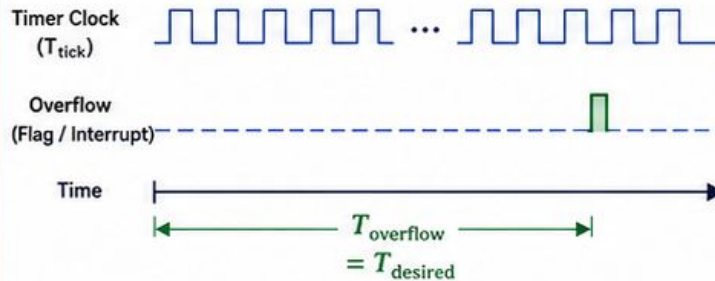
Relating desired time to timer overflow time

A Desired time equals overflow time (One overflow)

Key idea:

Program the timer so that one overflow occurs exactly at the desired time.

$$T_{\text{desired}} = T_{\text{overflow}}$$



Result: One overflow occurs at the desired time.

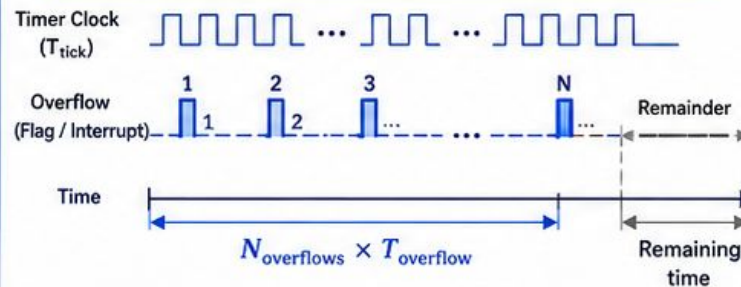
B Desired time is larger than overflow time (Multiple overflows)

Key idea:

Let the timer overflow multiple times, then handle the remaining time (if any).

$$N_{\text{overflows}} = \text{integer part of } \left(\frac{T_{\text{desired}}}{T_{\text{overflow}}} \right)$$

$$\text{Remaining time} = T_{\text{desired}} - (N_{\text{overflows}} \times T_{\text{overflow}})$$



Result: The timer overflows N times, then the remaining time is handled (e.g., by software or another timer mode).

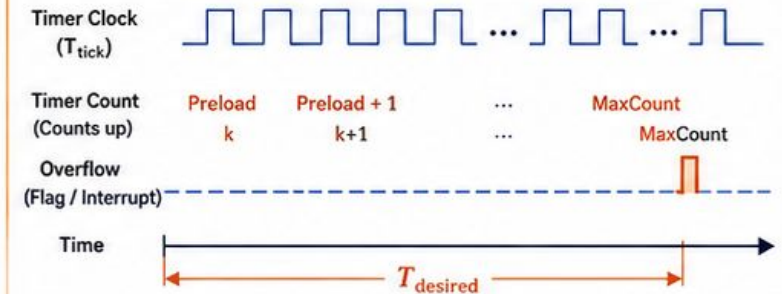
C Desired time is smaller than overflow time (Preload the timer)

Key idea:

Preload the timer with a starting count so that the overflow occurs earlier, at the desired time.

$$\text{Counts}_{\text{required}} = \frac{T_{\text{desired}}}{T_{\text{tick}}}$$

$$\text{Preload} = \text{MaxCount} + 1 - \text{Counts}_{\text{required}}$$

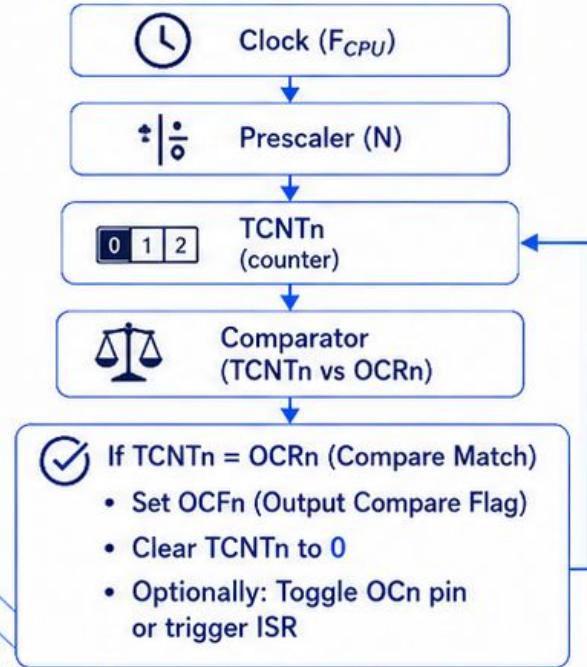


Result: Preloading makes the overflow occur sooner, exactly at the desired time.

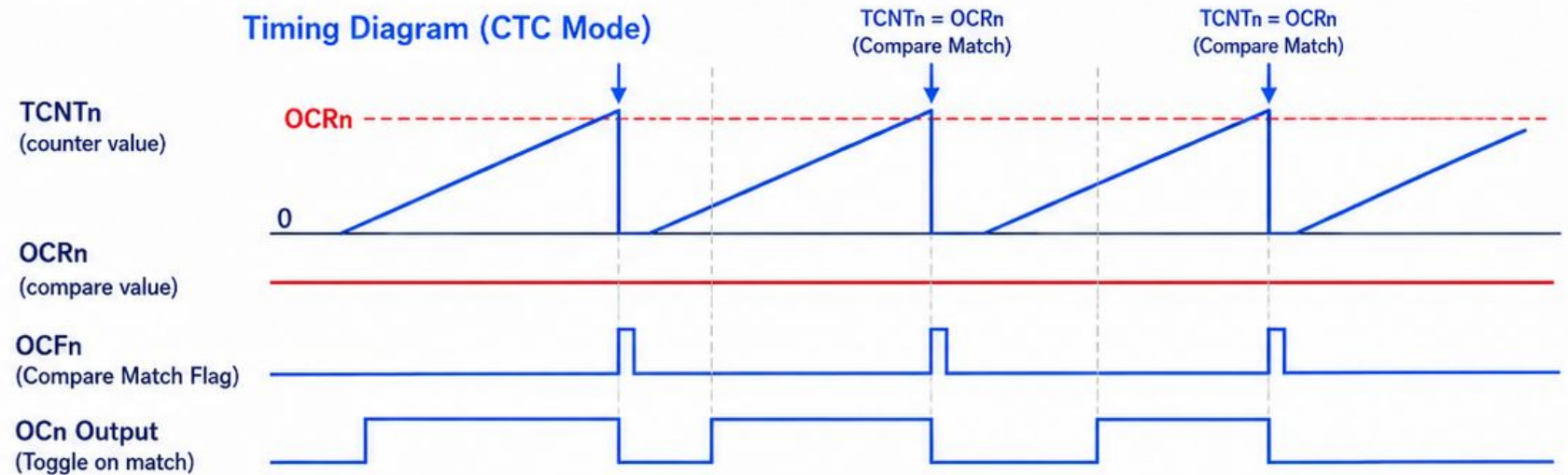
CTC Mode: Clear Timer on Compare Match

In CTC (Clear Timer on Compare Match) mode, the timer counts from 0 to OCR_n .
When $TCNT_n = OCR_n$, a compare match occurs, OCF_n is set, and the counter clears to 0.

CTC Operation Flow



Timing Diagram (CTC Mode)



Compare Interval (Time Between Matches)

$$T_{\text{compare}} = \frac{(OCR_n + 1) \times \text{Prescaler}}{F_{CPU}}$$

Compute OCR_n for Desired Interval

$$OCR_n = \left(\frac{T_{\text{desired}} \times F_{CPU}}{\text{Prescaler}} \right) - 1$$



Why CTC Mode?

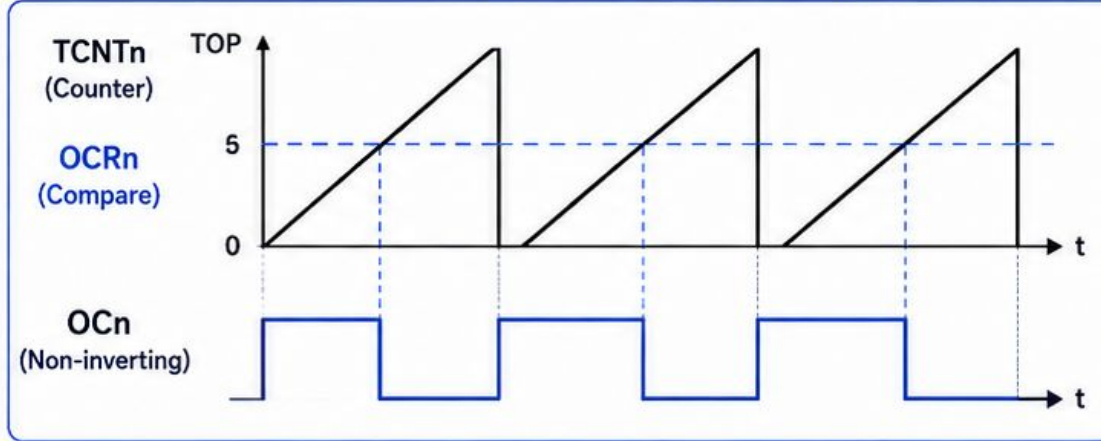
- No need to reload or preload the counter
- Automatic clear provides consistent timing
- Cleaner and more precise than using preload in Normal mode

Fast PWM Mode

Fast PWM is a single-slope PWM mode.
The counter counts from 0 to TOP
and then resets to 0 quickly.

- TCNTn counts up from 0 to TOP.
- When TCNTn = TOP, the counter resets to 0 on the next clock.
- Compare match (TCNTn = OCRn) controls the output.
- In non-inverting mode, OCn is set at BOTTOM and cleared on compare match.

Fast PWM Operation (Single-Slope)



8-bit Fast PWM (Mode 3)

TOP = 255 (0xFF)

$$f_{PWM} = \frac{F_{CPU}}{N \times 256}$$

Where:

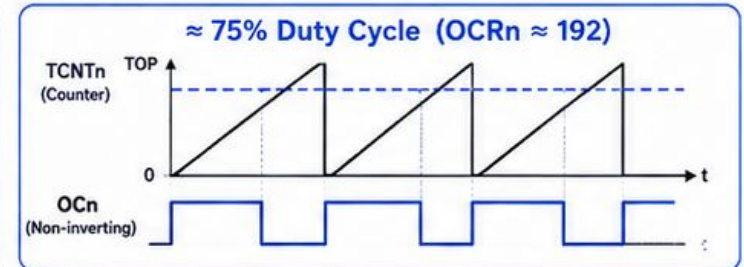
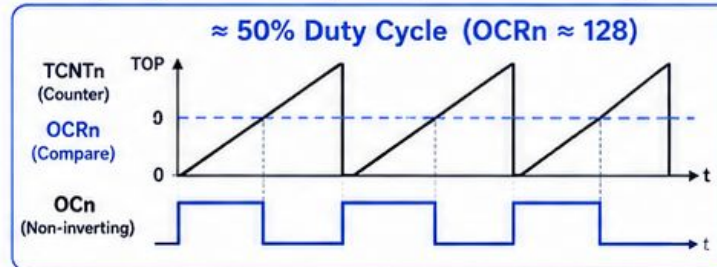
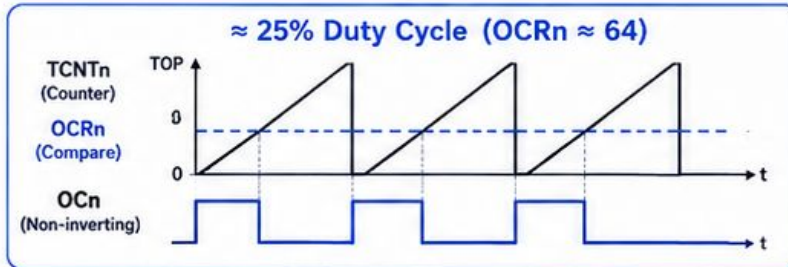
F_{CPU} = CPU clock frequency

N = Prescaler (1, 8, 64, 256, 1024)

Approximate Duty Cycle (Non-inverting mode)

$$\text{Duty (\%)} \approx \frac{OCRn}{255} \times 100\%$$

Duty Cycle Examples (Non-inverting Mode)



Applications



LED Brightness Control
Adjust LED intensity smoothly
by changing the duty cycle.



DC Motor Speed Control
Control average voltage
to set motor speed.

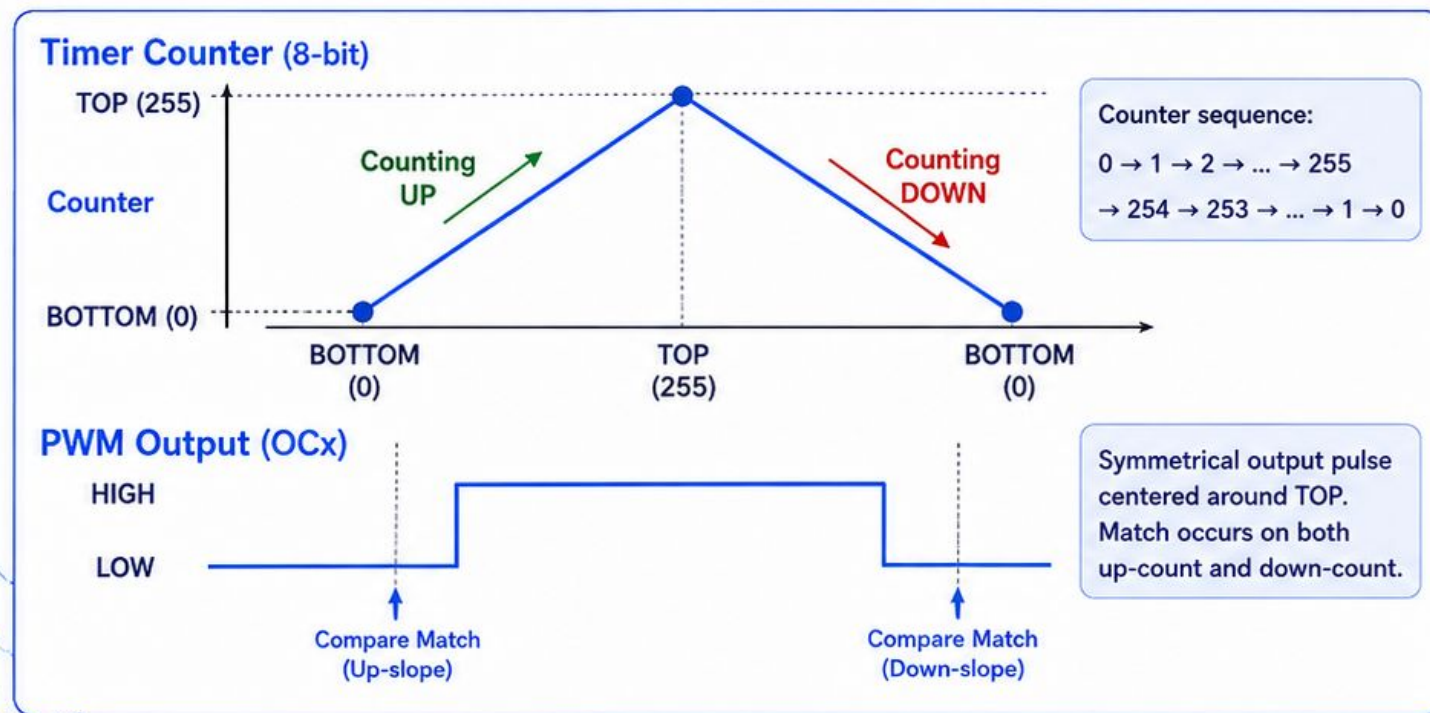


Buzzer / Power Control
Control tone, volume,
or power to a load.

Phase Correct PWM Mode

Dual-slope PWM with symmetrical output

The timer counter counts UP from BOTTOM to TOP, then counts DOWN from TOP to BOTTOM. The output compare match on both slopes creates a centered, symmetrical PWM waveform.



Comparison: Fast PWM vs. Phase Correct PWM

Fast PWM (Single-Slope)	Phase Correct PWM (Dual-Slope)
 Higher PWM frequency	 Lower PWM frequency
 Single-slope operation (counts up only)	 Dual-slope operation (counts up then down)
 Less symmetric switching	 More symmetric switching
 Better for maximum frequency applications	 Better for symmetry, lower ripple, and EMI reduction

8-bit Phase Correct PWM Frequency Formula

$$f_{\text{PWM}} = \frac{F_{\text{CPU}}}{N \times 510}$$

F_{CPU} : System clock frequency (Hz)
 N: Prescaler (1, 8, 64, 256, 1024)

Applications

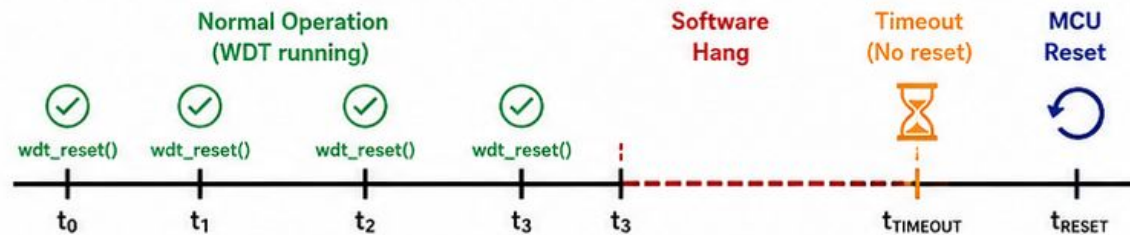


Phase Correct PWM is useful for motor control and power control applications where smoother symmetry is desired and torque ripple or EMI are important considerations.

WDT and ICU

Watchdog Timer (WDT)

WDT resets the MCU if software stops servicing (kicking) the watchdog.



WDTCR (Watchdog Timer Control Register)

Bit(s)	Name	Description
15	WDTOE	Watchdog Timer Output Enable
14	WDE	Watchdog System Enable
3:0	WDP2:0	Watchdog Prescaler (Timeout Period)

AVR C Example

```
#include <avr/wdt.h>

int main(void)
{
    wdt_enable(WDTO_2S); // ~2 second timeout
    while (1) {
        // Application code
        wdt_reset();      // Kick (service) WDT
    }
}
```

Key Points

- Regular `wdt_reset()` keeps the system alive.
- If not reset before timeout expires, the MCU is reset.

Key Bits

- WDTOE – Output enable
- WDE – System enable
- WDP2:0 – Timeout select

Input Capture Unit (ICU) – Timer1

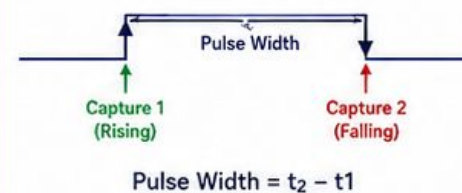
ICU captures the Timer/Counter1 value on selected edges of an external signal.



Measurement Examples

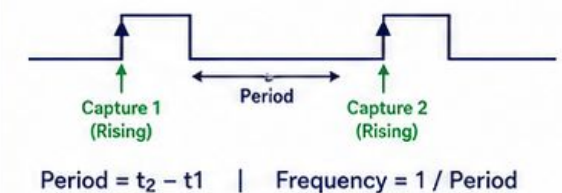
Pulse-Width Measurement

Capture rising edge, then falling edge.



Period / Frequency Measurement

Capture two consecutive rising edges.



Key Bits (TCCR1B / TIMSK1)

- ICES1 – Input Capture Edge Select (1 = rising, 0 = falling)
- ICNC1 – Input Capture Noise Canceler (1 = filter spikes)
- TICIE1 – Timer1 Input Capture Interrupt Enable

Key Takeaways

- Hardware timestamps: precise and low CPU load.
- Use ICU for timing, frequency, and pulse-width tasks.
- Works even with other CPU activities in the background.

Timer0 and Timer1 Registers Summary

Timer0 Registers (8-bit)

Register	Bit Fields	Function
TCCR0 (0x53)	FOC0 WGM00 COM01 COM00 WGM01 CS02:0	Waveform generation, Compare Output Mode, Force Output Compare, Clock Select
TCNT0 (0x52)	TCNT0[7:0]	8-bit Timer/Counter value
OCR0 (0x5C)	OCR0[7:0]	Output Compare Register 0
TIMSK (0x59)	- - - - TOIE0 OCIE0	Timer/Counter0 Interrupt Mask Register - TOIE0: Timer/Counter0 Overflow Interrupt Enable - OCIE0: Output Compare Match Interrupt Enable
TIFR (0x58)	- - - - TOV0 OCF0	Timer/Counter0 Interrupt Flag Register - TOV0: Timer/Counter0 Overflow Flag - OCF0: Output Compare Match Flag

Timer1 Registers (16-bit)

Register	Bit Fields	Function
TCCR1A (0x80)	COM1A1 COM1A0 COM1B1 COM1B0 FOC1A FOC1B WGM11 WGM10	Compare Output Mode, Force Output Compare, Waveform Generation (low bits)
TCCR1B (0x81)	ICNC1 ICES1 WGM13 WGM12 CS12 CS11 CS10	Input Capture Noise Canceller, Edge Select, Waveform Generation (high bits), Clock Select
TCNT1H/L (0x85/0x84)	TCNT1[15:0]	16-bit Timer/Counter1 value
OCR1AH/L (0x89/0x88)	OCR1A[15:0]	Output Compare Register 1 A
OCR1BH/L (0x8B/0x8A)	OCR1B[15:0]	Output Compare Register 1 B
ICR1H/L (0x87/0x86)	ICR1[15:0]	Input Capture Register 1
TIMSK (0x6F)	- - TOIE1 OCIE1A OCIE1B TICIE1 - -	Timer/Counter1 Interrupt Mask Register - TOIE1: Timer/Counter1 Overflow Interrupt Enable - OCIE1A: Output Compare A Match Interrupt Enable - OCIE1B: Output Compare B Match Interrupt Enable - TICIE1: Input Capture Interrupt Enable
TIFR (0x36)	- - TOV1 OCF1A OCF1B ICF1 - -	Timer/Counter1 Interrupt Flag Register - TOV1: Timer/Counter1 Overflow Flag - OCF1A: Output Compare A Match Flag - OCF1B: Output Compare B Match Flag - ICF1: Input Capture Flag



Note: Timer1 is 16-bit. When reading or writing 16-bit registers (TCNT1, OCR1A/B, ICR1), access high byte first, then low byte.